Neural Models: Learning, Depth, Activations, and Output Layers

Lab Report

Shrihari Karthik (f20230868)
BITS Pilani, Goa

Accompanying code: xor_lab.ipynb (PyTorch, CPU)

All results below come from a single seeded configuration: seed = 9, Adam optimizer, learning rate = 0.1, and 4000 full-batch training steps. The final cell of the notebook reprints the quantities reported here, so the results are reproducible by running the notebook from beginning to end.

1 Task 1 – Problem Specification

1.1 Input / Output Spaces

The agent observes two redundant binary sensors and must raise a disagreement warning. The input space is

$$X = \{0, 1\}^2,$$

which consists of exactly four possible inputs, while the label space is

$$Y = \{0, 1\}.$$

The model produces one real-valued logit z. The probability of class 1 is

$$p = \sigma(z) = \frac{1}{1 + e^{-z}},$$

and the final label is obtained by thresholding the probability at 0.5.

1.2 The Four Labelled Examples

x_1	x_2	y (warning)	Reading
0	0	0	Both sensors agree (inactive)
0	1	1	Sensors disagree
1	0	1	Sensors disagree
1	1	0	Both sensors agree (active)

Table 1: The four XOR training examples.

1.3 XOR Diagram

Figure 1 shows the XOR points in the $x_1 - x_2$ plane together with the learned hidden representation.

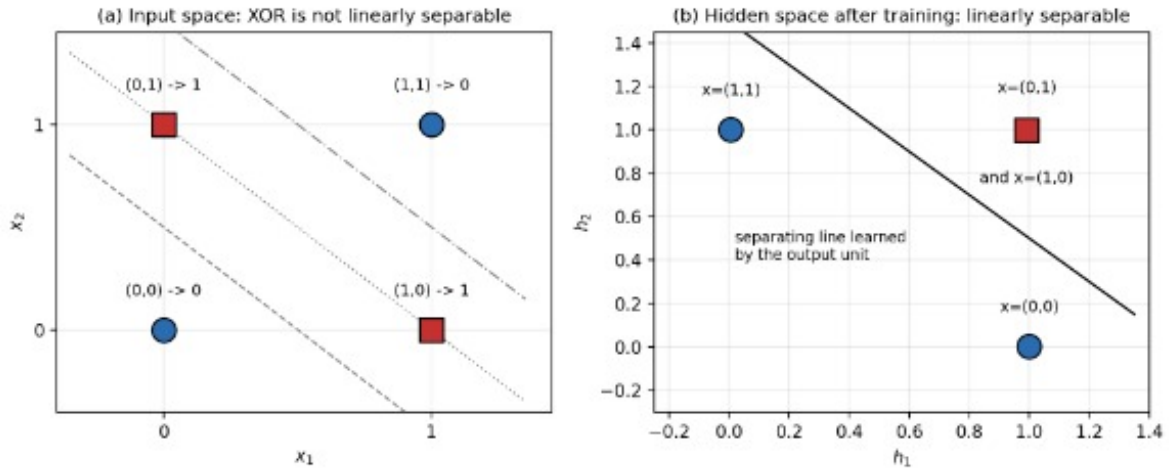


Figure 1: XOR input space and learned hidden representation. The XOR classes are not linearly separable in the original input space, while the nonlinear hidden layer produces a representation that can be separated by the output unit.

1.4 Why One Straight Decision Boundary Cannot Work

A linear classifier assigns the score

$$z = w_1 x_1 + w_2 x_2 + b$$

and divides the plane into the regions $z > 0$ and $z < 0$.

For XOR, we require

$$z(0,1) > 0, \qquad z(1,0) > 0,$$

while simultaneously requiring

$$z(0,0) < 0, \qquad z(1,1) < 0.$$

Adding the first pair gives

$$w_1 + w_2 + 2b > 0,$$

while adding the second pair gives

$$w_1 + w_2 + 2b < 0.$$

These requirements contradict one another. Therefore, no choice of (w_1, w_2, b) can correctly classify all four XOR points.

Geometrically, the class-0 points occupy one diagonal of the square and the class-1 points occupy the other diagonal. The classes are therefore interleaved rather than linearly separable.

1.5 Prediction for a Single Affine Layer + Sigmoid

I predicted that a single affine transformation followed by a sigmoid would fail to solve XOR.

Since no linear boundary can separate the classes, the model should settle near the solution that predicts 0.5 for every example. The corresponding binary cross-entropy is

$$\ln 2 \approx 0.693147.$$

The prediction was therefore that all four probabilities would be close to 0.5, giving only 2/4 correct classifications under tie-breaking.

The experiment confirmed this prediction. The initial loss was 0.712528, while the final loss was 0.693147, with probabilities

$$[0.5001,\ 0.5001,\ 0.5001,\ 0.5002].$$

The learned weights were approximately

$$W = [0.00024,\ 0.00024], \qquad b = 0.00025.$$

Adding further affine layers would not solve the problem, because a composition of affine transformations is still an affine transformation.

1.6 Think About It

With only four points, XOR tests a claim about *representation* rather than data volume or model capacity. A model can contain many parameters and still be affine in x. In that case, the XOR target lies outside the model's hypothesis class.

2 Task 2 – Model Design

The baseline architecture is

$$\boxed{\text{2 inputs} \to \text{2 hidden units} \to \text{1 output}}$$

with a nonlinear hidden activation.

Component	**Choice**	**Reason**
Architecture	2–2–1	Two inputs, two nonlinear hidden units, and one output logit.
Hidden activation	Sigmoid (baseline); Tanh and ReLU compared	Breaks the affine collapse. The baseline experiment uses sigmoid, while Task 4 compares the three activations.
Output	One logit, sigmoid applied inside the loss	The task requires one yes/no answer. Sigmoid maps the real-valued logit into a probability.
Loss	BCEWithLogitsLoss	Matches a Bernoulli output and combines sigmoid and binary cross-entropy in a numerically stable implementation.
Optimiser	Adam, learning rate = 0.1, 4000 full-batch steps	The dataset contains only four points, so full-batch training is exact and Adam converges reliably at this scale.
Initialisation	PyTorch default random uniform, seed = 9	Random values break the symmetry between the two hidden units.

Table 2: Model design and engineering choices.

2.1 Why the Hidden Nonlinearity Is Scientifically Necessary

Composing two affine layers gives

$$W^{(2)}\left(W^{(1)}x + b^{(1)}\right) + b^{(2)} = \left(W^{(2)}W^{(1)}\right)x + \left(W^{(2)}b^{(1)} + b^{(2)}\right),$$

which is again a single affine transformation.

Therefore, any depth consisting only of affine layers has the same representational power as one affine layer.

The elementwise nonlinear activation prevents this collapse. It allows the hidden layer to transform the input space so that the classes become linearly separable.

Thus, depth by itself is not the essential ingredient; **nonlinearity is**.

2.2 Why Sigmoid + Binary Cross-Entropy Is Appropriate

The target is one Bernoulli variable, so the natural output is a probability. The sigmoid function maps an unbounded logit into the interval $(0, 1)$.

Binary cross-entropy is the negative log-likelihood of the Bernoulli model. Thus, minimising it corresponds to maximum likelihood.

For a logit z,

$$\frac{\partial L}{\partial z} = p - y.$$

In PyTorch, `BCEWithLogitsLoss` is preferred over explicitly applying sigmoid followed by `BCELoss`, because it evaluates the same objective using a numerically stable implementation.

2.3 Validation Criteria

The following checks were fixed before running the main experiment.

#	Pass condition
1	Final loss is well below $\ln 2 = 0.6931$.
2	All four labels are correctly classified: $[0, 1, 1, 0]$.
3	Probabilities are decisive and close to 0 or 1 rather than 0.5.
4	Gradients are nonzero early and approach zero at convergence.
5	The result is reproducible under a fixed seed and succeeds for a majority of seeds.

Table 3: Validation criteria fixed before execution.

A nonzero gradient alone does not demonstrate successful learning. The loss, predictions, probabilities, and repeatability checks are needed to establish that the learning signal is actually useful.

2.4 Think About It: Hidden Units Without Targets

The hidden units are not assigned target values. Backpropagation determines what they should compute through the chain rule.

For example,

$$\frac{\partial L}{\partial h} = W^{(2)T}(p - y).$$

The output weights therefore act as the credit-assignment channel. The hidden features emerge because they are useful for reducing the output loss.

3 Task 3 – LLM Usage

3.1 Exact Prompt Used

The following prompt was supplied to the LLM:

> Generate minimal PyTorch code for the following model and dataset. Do not change the architecture or the task.
>
> Dataset (full batch, all four examples explicit): $X = [[0, 0], [0, 1], [1, 0], [1, 1]]$, $y = [0, 1, 1, 0]$ (XOR: two redundant binary sensors, warn when exactly one is active).
>
> Model: 2 inputs $\rightarrow$ 2 hidden units with a sigmoid hidden activation $\rightarrow$ 1 output.
>
> Output/loss: produce a raw logit and use `nn.BCEWithLogitsLoss` (do NOT apply `nn.Sigmoid` and then `BCELoss`). Random weight initialisation. Full-batch training, a few thousand lightweight CPU steps, Adam.

After training, report: (1) the initial and final loss; (2) all four predicted probabilities; (3) the thresholded labels; (4) at least one parameter-gradient tensor obtained after `backward()`. Set a random seed for reproducibility and explain each test in one sentence. Keep it to one file, no plotting.

3.2 Task 5 Follow-up Prompt

For the three-class extension, the following follow-up prompt was used:

> Modify ONLY the output layer and the loss of the previous code so the same two inputs are classified into three classes: class 0 = $(0, 0)$, class 1 = $(0, 1)$ or $(1, 0)$, class 2 = $(1, 1)$.
>
> Keep the 2-unit hidden layer unchanged. Use three logits and `nn.CrossEntropyLoss` with integer class targets. After training, print the softmax probability vector for all four inputs and verify that each vector sums to 1.

3.3 What the Generated Code Contained

The first draft was structurally correct. It contained an

```
nn.Sequential(Linear(2,2), Sigmoid(), Linear(2,1))
```

architecture, `BCEWithLogitsLoss`, an Adam optimiser, and the standard training loop:

```
opt.zero_grad()
logits = model(X)
loss = lossf(logits, Y)
loss.backward()
opt.step()
```

The five steps correspond to:

1. clear old gradients;
2. perform the forward pass;
3. calculate the scalar loss;
4. perform reverse-mode automatic differentiation;
5. update the parameters.

3.4 Corrections and Changes

#	Change	Reason
1	Targets reshaped to a $(4, 1)$ floating-point column	The original one-dimensional target could broadcast against the $(4, 1)$ logits and silently produce an incorrect 4×4 loss matrix.
2	Added a second forward/backward pass after training	The original gradient was from the last pre-update parameters rather than the final trained parameters.
3	Changed seed from 0 to 9 and increased training to 4000 steps	At seed 0, tanh and ReLU stalled. Seed and step count are legitimate engineering settings because the scientific task remained unchanged.
4	Added a per-example gradient-average check and finite-difference check	These independently verify the mean-loss gradient and autograd calculation.
5	Added the identical-initialisation experiment	The zero-weight experiment alone creates a completely dead network. The additional identical nonzero variant isolates the symmetry problem.
6	Added assertions for softmax row sums	This turns the probability-normalisation check into an executable test rather than relying on visual inspection.

Table 4: Corrections and engineering changes made to the generated code.

3.5 Brief Code Inspection

The forward pass occurs in

```
model(X)
```

which performs the sequence

$$\text{Linear} \to \text{activation} \to \text{Linear}.$$

The scalar loss is calculated using

```
lossf(logits, Y)
```

and is averaged over the four examples.
Reverse-mode automatic differentiation occurs at

```
loss.backward()
```

which computes gradients for the parameters.
Finally,

```
opt.step()
```

is the operation that changes the parameters.
`opt.zero_grad()` is necessary because PyTorch accumulates gradients by default.

4 Task 4 – Experimental Results

4.1 A. Initial vs Final Loss

Model	Initial Loss	Final Loss	Verdict
Affine only + sigmoid (control)	0.712528	0.693147	Stuck at $\ln 2$
2–2–1, sigmoid hidden	0.693800	0.000088	Solved

Table 5: Initial and final losses.

4.2 B. Four Predictions

x_1	x_2	Target y	$P(y=1)$	Predicted
0	0	0	0.000108	0
0	1	1	0.999931	1
1	0	1	0.999931	1
1	1	0	0.000107	0

Table 6: Predictions of the trained binary XOR model.

All four examples were classified correctly.

The hidden representation produced two useful features. The two hidden units behaved approximately like OR and AND respectively:

$$h(0,0) = (1.000, 0.003),$$

$$h(0,1) = (0.992, 0.998),$$

$$h(1,0) = (0.992, 0.998),$$

$$h(1,1) = (0.006, 1.000).$$

The output unit can then separate these transformed points with one linear boundary.

4.3 C. Backpropagation Gradient

The quantity

```
model[0].weight.grad
```

contains

$$\frac{\partial L}{\partial W^{(1)}}.$$

It has the same 2×2 shape as the first-layer weight matrix.

Using the chain rule,

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^T,$$

where

$$\delta^{(1)} = \left(W^{(2)T}(p-y)\right) \odot f'(a^{(1)}).$$

The measured gradient values were:

Quantity	Value
$\frac{\partial L}{\partial W^{(1)}}$ at trained parameters	$\begin{bmatrix} 3.24 \times 10^{-7} & 4.13 \times 10^{-7} \\ -7.79 \times 10^{-7} & -6.27 \times 10^{-7} \end{bmatrix}$
$\frac{\partial L}{\partial W^{(1)}}$ at step 10	$\begin{bmatrix} 1.99 \times 10^{-4} & 3.09 \times 10^{-4} \\ -6.65 \times 10^{-3} & -5.23 \times 10^{-3} \end{bmatrix}$
Mean-loss gradient vs average of per-example gradients	Maximum absolute difference $< 10^{-7}$
Autograd vs finite difference	Agreement to 7 decimal places

Table 7: Gradient checks.

4.4 Why the Mean Loss Gives the Average Gradient

The loss is

$$L = \frac{1}{4} \sum_{n=1}^{4} L_n.$$

Differentiation is linear, so

$$\frac{\partial L}{\partial W} = \frac{1}{4} \sum_{n=1}^{4} \frac{\partial L_n}{\partial W}.$$

Therefore, the gradient of the mean loss is exactly the mean of the four example-wise gradients. The experiment verified this numerically.

The near-zero gradient at convergence is evidence of convergence, rather than evidence of a broken computation graph. At training step 10, the gradient was several orders of magnitude larger.

4.5 D. Zero-Initialisation Symmetry Experiment

Two variants were tested.

4.5.1 Variant 1 – All Weights and Biases Set to Zero

The network never moved:

Step	0	1	2	10	100	1000	Final
Loss	0.693147	0.693147	0.693147	0.693147	0.693147	0.693147	0.693147
$\|\partial L/\partial W^{(1)}\|$	0	0	0	0	0	0	0
Rows of $W^{(1)}$	Identical	Identical	Identical	Identical	Identical	Identical	Identical

Table 8: All-zero initialisation experiment.

The reason is structural. Backpropagation sends

$$\frac{\partial L}{\partial h} = W^{(2)T}(p - y)$$

into the hidden layer. Since

$$W^{(2)} = 0,$$

the hidden layer receives exactly zero learning signal.
Consequently,

$$\frac{\partial L}{\partial W^{(1)}} = 0$$

and the first layer cannot move away from the origin.

Every hidden unit also produces

$$f(0) = 0.5.$$

The output can therefore only learn the base rate, leaving the loss at

$$\ln 2 = 0.693147.$$

4.5.2 Variant 2 – Identical but Non-Zero Initialisation

The two hidden units were initialized as exact twins. Their incoming weights, biases, and outgoing weights were identical.

Step	0	1	2	10	100	3999
Loss	0.695184	0.699067	0.694058	0.693655	0.484871	0.477398
$\max \lvert \text{row}_0 - \text{row}_1 \rvert$	0	0	0	0	0	0

Table 9: Identical nonzero initialisation experiment.

The two rows remain exactly identical for all training steps.

Identical hidden units compute identical outputs and therefore receive identical upstream derivatives. They consequently receive identical gradients and identical parameter updates.

The symmetry is therefore preserved by gradient descent.

The final probabilities were approximately

$$[0.00003,\ 0.66666,\ 0.66666,\ 0.66667],$$

with predictions

$$[0, 1, 1, 1].$$

Thus only 3/4 examples were classified correctly.

Random initialization is therefore important because it provides the initial asymmetry that allows the two hidden units to learn distinct features.

4.6 E. Activation Comparison

The three activation functions were tested using the same seed, optimizer, learning rate, and number of training steps.

Activation	**Final Loss**	**4/4 Correct?**	**Early Gradient Norm**	**Successful Seeds**
Sigmoid	0.000088	Yes	0.008466	8/12
Tanh	0.000018	Yes	0.046471	7/12
ReLU	0.000010	Yes	0.106251	1/12

Table 10: Activation-function comparison. Gradient norm was measured at training step 10.

The step-10 gradient tensors were:

$$\text{Sigmoid: } \begin{bmatrix} 1.99 \times 10^{-4} & 3.09 \times 10^{-4} \\ -6.65 \times 10^{-3} & -5.23 \times 10^{-3} \end{bmatrix},$$

$$\text{Tanh: } \begin{bmatrix} -4.09 \times 10^{-3} & -7.25 \times 10^{-3} \\ -3.88 \times 10^{-2} & -2.42 \times 10^{-2} \end{bmatrix},$$

$$\text{ReLU: } \begin{bmatrix} -1.04 \times 10^{-1} & 0 \\ 6.70 \times 10^{-3} & -1.92 \times 10^{-2} \end{bmatrix}.$$

4.6.1 Interpretation

The early gradient norm increases from sigmoid to tanh to ReLU:

$$0.0085 \to 0.0465 \to 0.1063.$$

Scientifically, this follows from the derivatives contributed by each activation during backpropagation. Sigmoid has derivative at most 0.25 and becomes small away from zero. Tanh has derivative at most 1, while an active ReLU has derivative exactly 1.

However, the engineering result is different. For seed 9, all three activations solved the four-point problem. The final losses were all extremely small, so their differences are not meaningful on such a tiny dataset.

The seed sweep provides a more interesting result. ReLU had the largest gradients but succeeded on only 1/12 seeds, whereas sigmoid succeeded on 8/12 and tanh on 7/12.

For ReLU, a hidden unit whose pre-activation is negative for every training example has derivative exactly zero. Such a unit becomes dead and cannot receive gradient through its incoming weights.

Therefore, a large gradient does not necessarily imply reliable learning, and no activation can be declared universally best from this four-point experiment.

4.6.2 Saturation versus ReLU Death

A saturated sigmoid and a dead ReLU can both produce small gradients, but the mechanisms are different.

For a saturated sigmoid,

$$|a| \text{ is large}, \qquad h \approx 0 \text{ or } 1,$$

and

$$f'(a) = h(1-h)$$

is small but still strictly positive.

For a dead ReLU,

$$a < 0,$$

so

$$h = 0, \qquad f'(a) = 0.$$

Therefore, examining the pre-activation values allows the two cases to be distinguished. A column that is negative for every input is a signature of a dead ReLU unit.

5 Task 5 – Three-Class Extension

5.1 New Class Definition

The same sensor inputs were relabelled into three mutually exclusive classes:

Class	Meaning	Input
0	Both sensors inactive	$(0, 0)$
1	Sensors disagree	$(0, 1), (1, 0)$
2	Both sensors active	$(1, 1)$

Table 11: Three-class sensor problem.

The hidden representation remains important because the three-class problem is still not linearly separable in the original input space.

5.2 Modified Architecture

Only the output layer and loss were changed.

The final layer changed from

`Linear(2,1)`

to

`Linear(2,3)`.

The loss changed from

`BCEWithLogitsLoss`

to

`CrossEntropyLoss`.

The hidden layer remained unchanged.

The final weight matrix therefore has shape

$$3 \times 2,$$

with a bias vector of length 3.

Each input produces exactly three logits. The resulting logit tensor for the four examples has shape

$$(4, 3).$$

5.3 Why Softmax Probabilities Sum to One

The softmax function is

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Every component is divided by the same positive normalising constant. Therefore,

$$\sum_k \text{softmax}(z)_k = \frac{\sum_k e^{z_k}}{\sum_j e^{z_j}} = 1.$$

Hence the three outputs form a valid probability distribution.

5.4 Why the Logit Gradient Is $p - y$

For one-hot targets,

$$L = -\sum_k y_k \log p_k.$$

Since

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}},$$

differentiating through the softmax and cross-entropy gives

$$\boxed{\frac{\partial L}{\partial z_k} = p_k - y_k}$$

The gradient can be interpreted as a signed probability error. The true class is pushed upward according to how much probability it is missing, while the incorrect classes are pushed downward according to the probability they currently hold.

5.5 Predicted Probability Vectors

x_1	x_2	True	p_0	p_1	p_2	Predicted
0	0	0	0.999992	0.000008	0.000000	0
0	1	1	0.000003	0.999995	0.000003	1
1	0	1	0.000003	0.999995	0.000003	1
1	1	2	0.000000	0.000011	0.999989	2

Table 12: Predicted probability vectors for the three-class model.

The cross-entropy loss decreased from

$$1.147700$$

to

$$0.000007.$$

All four inputs were classified correctly.
For $x = (0, 1)$, the probability vector was

$$[0.000003,\ 0.999995,\ 0.000003].$$

Its components sum to

$$1.000000000000,$$

and the notebook verified the corresponding condition for all four inputs.

5.6 Optional Shift-Invariance Diagnostic

Adding a constant c to every logit does not change the softmax:

$$\text{softmax}(z + c) = \text{softmax}(z).$$

This occurs because the common factor e^c cancels between the numerator and denominator.
For numerical stability, implementations subtract the largest logit before exponentiation:

$$z'_k = z_k - \max_j z_j.$$

This ensures that the largest exponent is

$$e^0 = 1,$$

preventing overflow.

6 Reflection Questions

6.1 Q1. What did the XOR experiment demonstrate about depth versus nonlinearity?

Depth and nonlinearity are not interchangeable. Stacking affine layers only multiplies matrices and produces another affine map. The affine-only control remained at

$$\ln 2 = 0.693147$$

with probabilities near 0.5.

Adding one nonlinear hidden layer with only two units reduced the loss to 0.000088 and solved XOR. Thus, the important ingredient is the nonlinearity between layers; depth becomes useful because the nonlinearity prevents the affine collapse.

6.2 Q2. What evidence showed that backpropagation supplied a useful learning signal?

A nonzero gradient alone is insufficient because the identical-weight experiment had nonzero gradients but could only reach 3/4 accuracy.

The successful experiment provided stronger evidence:

- the loss decreased from 0.6938 to 8.8×10^{-5};
- all four labels became correct;
- probabilities became decisive rather than remaining near 0.5;
- the gradient magnitude decreased substantially as the model converged;
- the hidden layer learned two distinct features even though no hidden-layer targets were provided.

The final point is particularly strong evidence that backpropagation performed useful credit assignment.

6.3 Q3. Why did identical or zero initialisation prevent distinct hidden features?

There are two separate mechanisms.

With all-zero weights,

$$W^{(2)} = 0,$$

so

$$\frac{\partial L}{\partial h} = W^{(2)T}(p - y) = 0.$$

Therefore the first layer receives zero gradient and never changes.

With identical nonzero weights, the gradients are nonzero, but both hidden units compute exactly the same function. They consequently receive identical gradients and identical updates.

The symmetry therefore remains exact throughout training. Random initialisation breaks this symmetry and allows the two hidden units to learn different features.

6.4 Q4. How did the hidden activation affect the observed gradient?

Scientifically, the chain rule causes the backward pass to multiply the gradient by the derivative of the hidden activation. Sigmoid has a maximum derivative of 0.25, tanh has a maximum derivative of 1, and an active ReLU has derivative 1.

The measured step-10 gradient norms were

$$0.0085, \qquad 0.0465, \qquad 0.1063$$

for sigmoid, tanh, and ReLU respectively.

The engineering observation is separate. Although ReLU had the largest gradients, it succeeded on only 1/12 seeds. Sigmoid and tanh succeeded on 8/12 and 7/12 respectively.

Thus, larger gradients do not automatically imply more reliable learning.

6.5 Q5. Why must the output layer and loss be selected together?

The output layer and loss encode the probabilistic model of the target.

For a binary task,

$$\text{sigmoid} + \text{BCE}$$

models a Bernoulli variable.

For a one-of-K classification task,

$$\text{softmax} + \text{cross-entropy}$$

models a categorical distribution.

Matching the output and loss produces the clean gradient

$$\frac{\partial L}{\partial z} = p - y.$$

Using an inappropriate pairing can introduce unnecessary derivative factors or fail to produce a properly normalised class distribution.

6.6 Q6. Where did the LLM improve productivity, and where was human verification essential?

The LLM significantly reduced the amount of boilerplate coding required. It generated the model construction, Adam setup, training loop, probability reporting, gradient reporting, and the conversion to three logits with cross-entropy.

Human verification was essential because the first draft contained subtle problems that were not syntax errors. For example, the gradient printed after training corresponded to the pre-update parameters rather than the final trained parameters. The target shape also created a silent broadcasting issue.

These problems demonstrate why generated code must be tested rather than accepted simply because it runs.

6.7 Q7. Which tests would survive scaling up?

The following inexpensive tests should remain useful:

- monitoring the loss curve against a trivial baseline;
- gradient-norm monitoring for each layer;
- tensor-shape assertions;
- probability-normalisation checks;
- fixed-seed reproducibility;
- held-out accuracy.

Exhaustive finite-difference gradient checking becomes impractical because it requires multiple forward passes for each parameter. Similarly, enumerating every possible input only works for a tiny input space.

Testing many complete random seeds becomes expensive at scale, so a smaller number of runs and careful initialisation strategies become more appropriate. Inspecting individual hidden units manually is also replaced by aggregate activation statistics.

Conclusion

The XOR experiment demonstrated that nonlinear representation, rather than affine depth alone, is essential for representing XOR. The experiments also showed how backpropagation supplies the learning signal, why random initialisation breaks hidden-unit symmetry, and how activation functions influence gradient flow.

The three-class extension further demonstrated that the hidden representation can remain unchanged while the output layer and loss are adapted to a different prediction task. Finally, the LLM was useful for generating implementation boilerplate, but experimental verification remained essential for detecting subtle numerical and scientific errors.